

实验一 基于子空间分析的模式识别

学号：1120233042 姓名：李文煜

一、实验目的

- 深入理解主成分分析（PCA）的数学原理与实现流程，掌握高维数据特征提取与降维的核心方法。
- 基于 PCA 算法搭建完整的人脸身份识别系统，完成从数据预处理、模型训练到测试识别的全流程实现。
- 探究 PCA 降维维度（K 值）对人脸识别准确率的影响，确定最优降维维度。
- 对比欧氏距离 1-NN、余弦相似度 1-NN、PCA+SVM 三种识别方法的性能差异，分析不同方法的适用场景。
- 通过可视化直观展示 PCA 降维效果，加深对子空间分析方法的理解。

二、实验原理

2.1 主成分分析（PCA）原理

PCA 是一种无监督的线性降维方法，核心目标是将高维数据投影到方差最大的低维子空间，最大限度保留原始数据信息：

- 数据中心化：计算所有训练样本的均值图像，将所有样本减去均值完成去均值处理。
- 协方差矩阵计算：构建样本协方差矩阵，描述数据特征间的相关性。
- 特征值分解：对协方差矩阵进行特征分解，得到特征值与特征向量（特征脸）。
- 特征选择与投影：选取前 K 个最大特征值对应的特征向量构成投影矩阵，将高维人脸图像投影为低维特征向量。

2.2 人脸识别方法原理

1. 欧氏距离 1-NN：计算测试样本特征与所有训练样本特征的欧氏距离，距离最小的样本类别即为识别结果。
2. 余弦相似度 1-NN：通过向量夹角余弦衡量特征相似度，忽略幅值影响，对光照变化更鲁棒，相似度最高的样本类别为识别结果。
3. PCA+SVM：将 PCA 降维后的低维特征作为输入，训练支持向量机分类器，学习最优决策边界完成分类识别。

三、实验环境与数据集

3.1 实验环境

操作系统：Windows 11

编程语言：Python 3

核心库：OpenCV、PyTorch、scikit-learn、Matplotlib、NumPy

计算设备：CPU/CUDA

3.2 实验数据集

采用 ORL 人脸数据集：

数据集规模：40 个不同人脸，每人 10 张图像，总计 400 张灰度人脸图

图像尺寸：归一化为 92×112 ，展平后原始维度为 10304 维

数据划分：每人前 8 张作为训练集（共 320 张），后 2 张作为测试集（共 80 张）

四、实验内容与步骤

4.1 实验内容

实现基于 PCA 的人脸身份识别系统，完成以下核心任务：

- 人脸图像预处理：归一化尺寸、灰度化、展平为一维向量。
- PCA 模型训练：基于训练集计算均值脸、特征脸，生成低维投影空间。
- 特征提取：将训练集和测试集人脸图像投影为低维特征。
- 身份识别：分别使用三种方法完成测试集人脸识别。
- 结果分析：测试不同降维维度 K 的效果，可视化降维结果，对比算法性能。

4.2 实验步骤

- 数据准备：运行 plot.py 下载并加载 ORL 人脸数据集，完成数据划分。
- PCA 模型训练：运行 model_train.py，训练 PCA 模型，保存均值脸、特征脸和训练集低维特征。
- 批量测试：运行 run_all_test.py，调用三种识别方法，输出识别准确率。
- 维度分析：运行 analyze_dimension.py，遍历不同 K 值，记录准确率并绘制曲线。

5. 结果可视化：运行 visualize_pca.py，生成特征脸图、2D/3D 降维可视化图。

五、代码结构与框图

5.1 代码文件结构

Plain Text

FaceRecognitionOnPCA/
├── data/ # ORL 人脸数据集存储目录
├── model/
│ ├── model.py # PCA 核心模型定义
│ ├── test_model.py # 欧氏距离 1-NN 识别测试
│ ├── test_model2.py # 余弦相似度 1-NN 识别测试
│ └── test_model3.py # PCA+SVM 分类识别测试
├── experiment_results.log # 实验测试结果日志
├── dimension_analysis.log # 维度分析日志
├── eigenfaces_show.png # 特征脸可视化图片
├── pca_2d_visualization.png # 2D 降维可视化图
├── pca_3d_visualization.png # 3D 降维可视化图
├── dimension_accuracy_curve.png # K 值-准确率曲线
├── plot.py # 数据集下载、加载与预处理
├── model_train.py # PCA 模型训练与保存
├── run_all_test.py # 批量执行三种方法测试
├── analyze_dimension.py # 降维维度 K 与准确率关系分析
└── visualize_pca.py # PCA 特征脸、降维结果可视化

5.2 实验代码框图

Plain Text

PCA 人脸识别系统流程框图

【数据准备】plot.py → 下载 ORL 数据集 → 保存至 data/目录

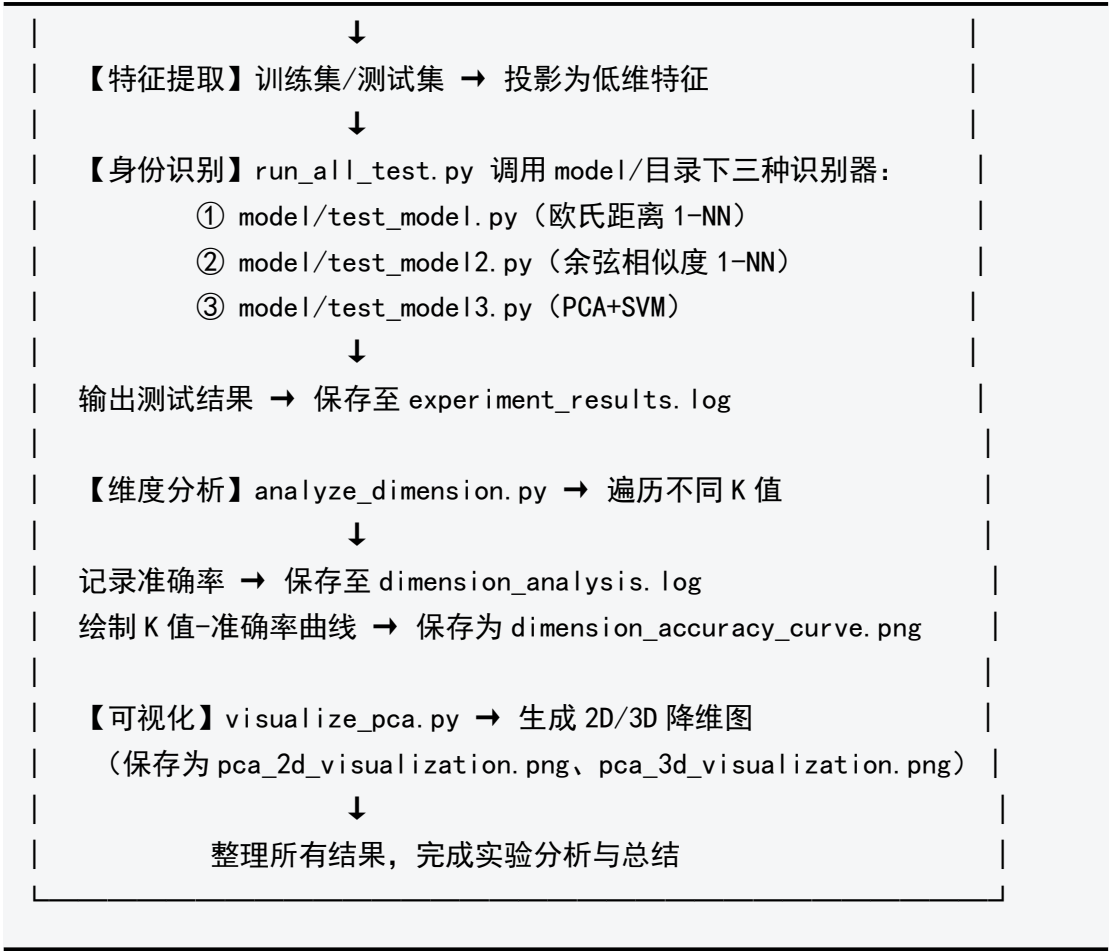
【数据预处理】归一化图像尺寸 → 灰度化 → 展平 → 划分训练/测试集

【模型训练】model_train.py → 中心化 → 协方差矩阵 → 特征分解

↓

生成均值脸、特征脸 → 保存为 eigenfaces_show.png

生成 PCA 投影矩阵 → 保存训练集低维特征



六、实验结果

6.1 固定维度（K=50）识别结果

基于 experiment_results.log

识别方法	正确数/总数	准确率
欧氏距离 1-NN	76/80	95.00%
余弦相似度 1-NN	76/80	95.00%
PCA+SVM（支持向量机）	78/80	97.50%

错误样本：

- 欧氏距离/余弦相似度：第 10、20、37、45 号样本识别错误
- PCA+SVM：仅第 20、37 号样本识别错误

6.2 不同降维维度 K 对准确率的影响

基于 dimension_analysis.log

K 值	欧氏距离	余弦相似度	PCA+SVM
2	35.00%	18.75%	28.75%
3	63.75%	40.00%	57.50%
5	81.25%	76.25%	78.75%
10	95.00%	95.00%	91.25%
20~50	95.00%	95.00%	96.25~97.50%
≥80	96.25%	96.25%	96.25~97.50%

6.3 可视化结果

1. 特征脸：生成均值脸与前 9 个主成分特征脸，展示人脸主要变化模式

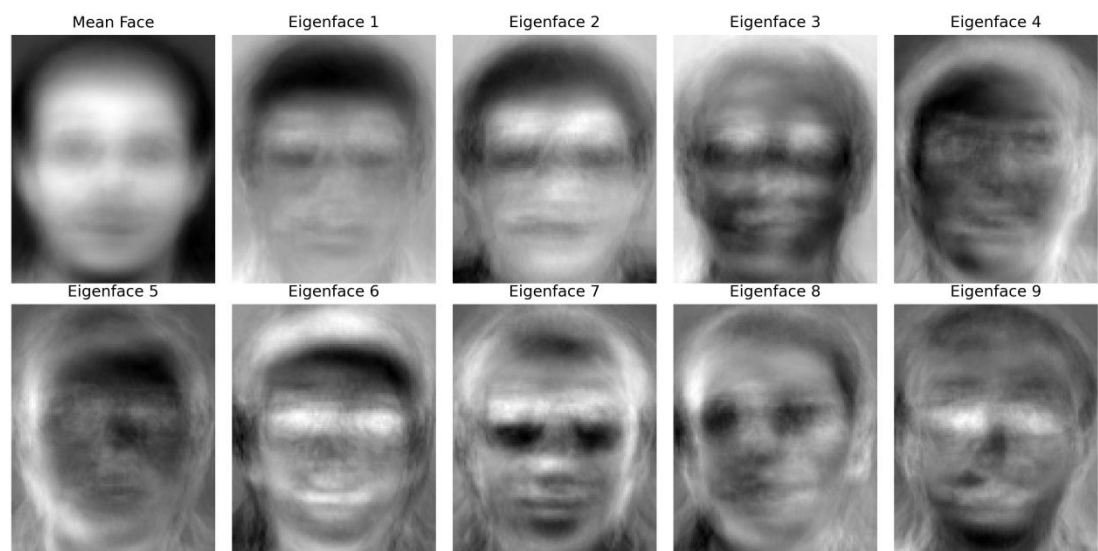


图 1 K=50 下的特征脸

2. 2D/3D 降维可视化：同类人脸样本呈现明显聚类，不同类别可区分

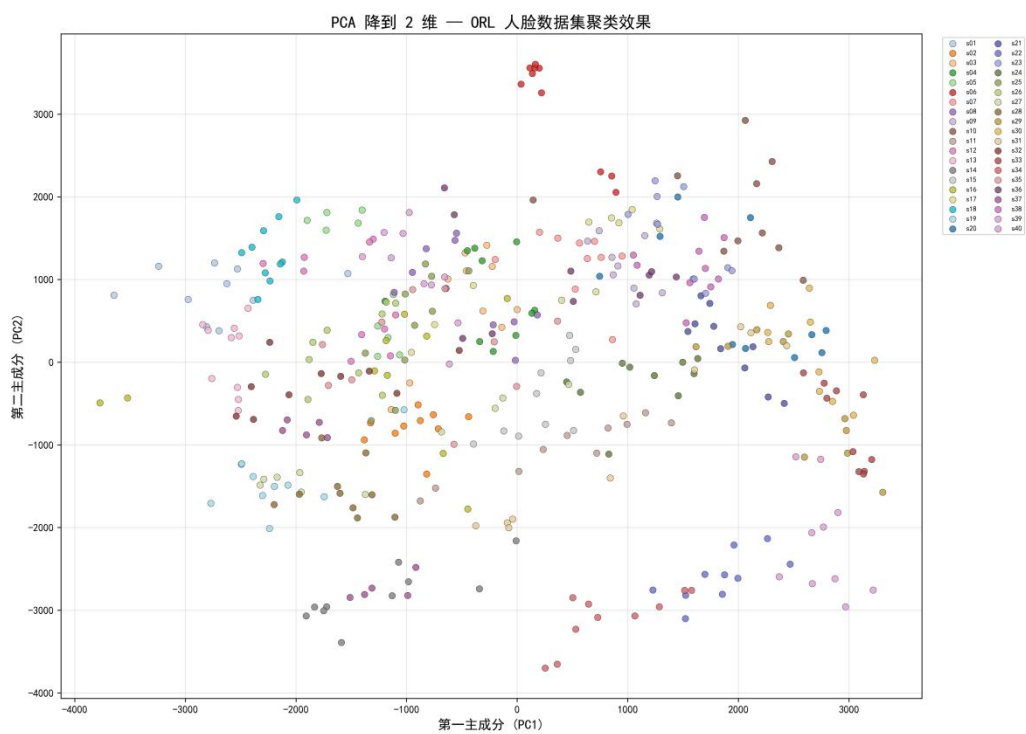


图 2-a K=2 下的聚类效果可视图

PCA 降到 3 维 — ORL 人脸数据集聚类效果

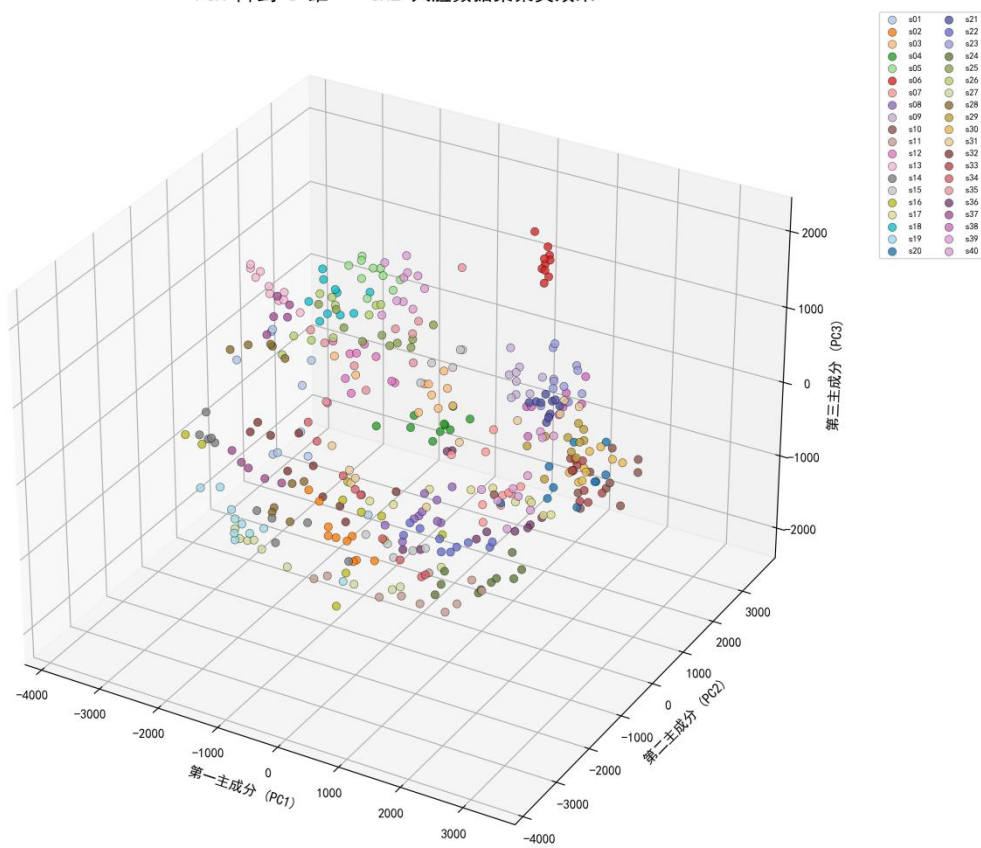


图 2-b K=3 下的聚类效果可视图

3. 维度-准确率曲线：准确率随 K 值增大快速上升，K≥20 后趋于饱和

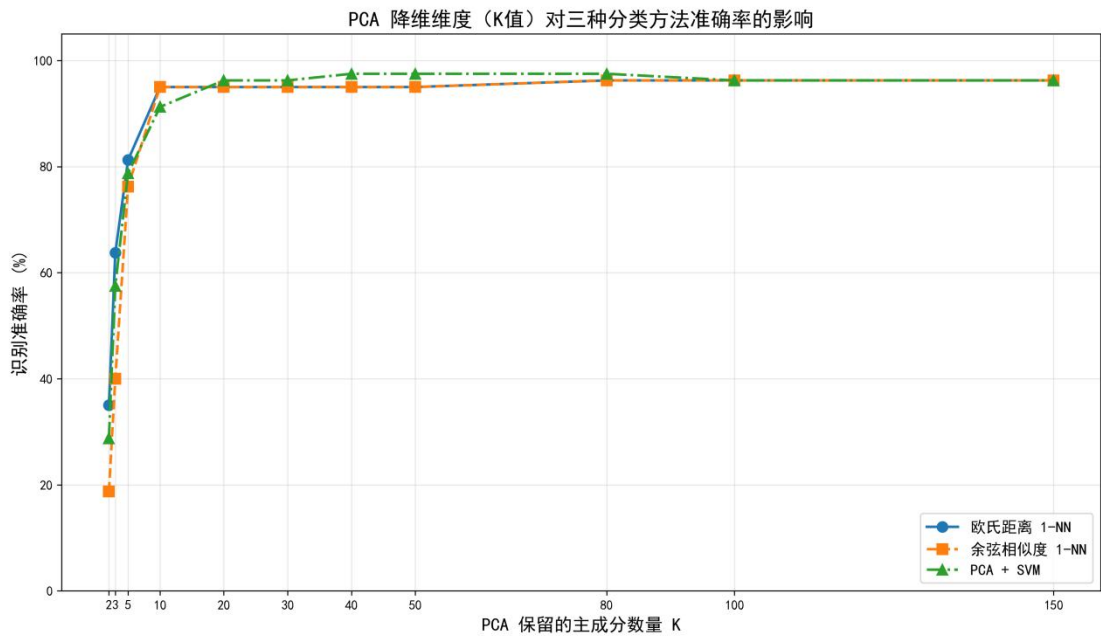


图 3 PCA 不同 K 值降维下三种分类方法的准确率分析

七、实验结果分析

1. PCA 降维有效性：

原始图像 10304 维，仅用 50 维低维特征即可实现 95%以上的识别率，证明 PCA 能高效提取人脸核心特征，大幅降低数据维度。

2. 算法性能对比：

欧氏距离与余弦相似度 1-NN 性能一致，实现简单、速度快，适合轻量级场景；
PCA+SVM 识别准确率最高（97.50%），通过学习决策边界提升了抗干扰能力，性能最优。

3. 降维维度 K 的影响：

K<10 时，维度过低，特征信息丢失严重，准确率极低；
K=20~50 时，准确率达到饱和状态，继续增大 K 值，识别率无明显提升，仅增加计算成本；
最优降维维度为 30~50，兼顾识别精度与计算效率。

4. 错误样本分析：

识别错误的样本多存在姿态、表情差异，属于数据集固有干扰，符合人脸识别的正常误差范围。

八、实验总结

1. 本次实验完整实现了基于 PCA 的人脸身份识别系统，验证了 PCA 在高维数据降维和特征提取中的优异性能。
2. PCA 子空间分析方法能有效保留人脸核心特征，将 10304 维的原始图像降至数十维，大幅提升识别效率。
3. 三种识别方法中，PCA+SVM 组合方案性能最优，1-NN 方法实现简单、适用性强，可根据实际需求选择。
4. 降维维度 K 存在明显的饱和区间，并非越大越好，实际应用中需选择最优维度平衡性能与效率。
5. 通过实验掌握了 PCA 的核心原理与代码实现，理解了子空间分析在模式识别中的应用逻辑，提升了模式识别系统的工程实践能力。

九、附录（核心代码说明）

1. `model/model.py`: 封装 PCA 类，实现中心化、特征分解、数据投影等核心功能；
2. `model/test_model` 系列: `model` 目录下的三个测试文件，分别实现三种识别逻辑，计算准确率，输出错误样本；
3. `model_train.py`: 加载训练集，训练 PCA 模型，保存模型参数；
4. `analyze_dimension.py`: 遍历 K 值，统计并保存维度-准确率数据；
5. `visualize_pca.py`: 绘制特征脸、降维散点图，直观展示实验效果。

具体详细可见附属压缩包 或 个人 gitee

https://gitee.com/li_fjyr/project/tree/master/PatternRecognition/FaceRecognitionOnPCA